

SIMATS ENGINEERING

Assessment Tool 2: Industry Problem Solving Task

Course Name	Cloud Computing and Big Data Analytics
Course Code	CSA15
Course Outcome	CO4 - Analyze big data characteristics and challenges across domains including security and applic
Dave's Taxonomy	Articulation (Level 4)
Question Count	5
Total Marks	40
Duration	60 Minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1: Retail Data Optimization Problem [8 Marks]

Industry Context

A retail chain running both physical stores and an online platform generates high-velocity, high-volume, and highly varied data: point-of-sale transactions, RFID/inventory feeds, clickstream and browsing logs, app usage, loyalty-card history, and social/marketing engagement data. The business goals are (a) real-time inventory visibility across stores and warehouses, (b) accurate demand forecasting to avoid stock-outs and overstocking, and (c) personalized marketing driven by customer behaviour. This is a classic omni-channel retail big data problem requiring both streaming and batch analytics working together.

Proposed Big Data Architecture (Lambda-style)

- **Ingestion Layer:** POS systems, RFID/IoT shelf sensors, mobile app and website clickstreams, and supplier/warehouse feeds are ingested using **Apache Kafka** (or AWS Kinesis) as a durable, high-throughput message bus that decouples producers from consumers.
- **Speed/Stream Layer:** **Apache Spark Structured Streaming** (or Apache Flink) consumes the Kafka topics to update real-time inventory counts, detect low-stock alerts, and score in-session recommendations with sub-second to few-second latency.
- **Batch/Storage Layer:** Raw and curated data is stored in a cloud **Data Lake** (Amazon S3 / Azure Data Lake Storage) using a Bronze-Silver-Gold (medallion) structure, queried via **Apache Hive/Delta Lake** for structured analytics and long-term trend analysis.
- **Serving Layer:** A NoSQL store such as **Apache Cassandra** or **DynamoDB** holds low-latency current-state inventory for store/app lookups; a **Data Warehouse** (Snowflake/Redshift/BigQuery) supports BI dashboards.
- **Analytics/ML Layer:** Demand forecasting uses time-series models (ARIMA/Prophet) and gradient-boosted or LSTM models trained via **Spark MLlib / TensorFlow** on historical sales, seasonality, weather and promotion data. Personalization uses collaborative filtering and recommendation engines (e.g., Spark ALS) plus clickstream-based propensity models.
- **Orchestration & Visualization:** **Apache Airflow** schedules batch ETL/forecast retraining jobs; **Power BI/Tableau** and mobile dashboards surface inventory KPIs and campaign performance to managers.

Solution Reasoning

The architecture separates a fast streaming path (real-time inventory and immediate recommendations) from a batch path (deep forecasting and model training), which is scalable because Kafka and Spark scale horizontally with data volume, and practical because retailers can start with core POS/inventory streams and progressively add clickstream and IoT sources.

Security / Risk Considerations

- Encrypt customer PII (loyalty IDs, payment tokens) at rest and in transit; apply field-level tokenization for payment data to remain PCI-DSS compliant.
- Role-based access control (RBAC) on the data lake/warehouse so store staff, marketing, and finance see only authorized data; audit logging of all access.
- Data quality/latency monitoring to prevent stale inventory counts from causing incorrect stock or pricing decisions; graceful degradation if the streaming pipeline lags.

Question 2: Social Media Fraud Detection Problem [8 Marks]

Industry Context

A social media platform handling billions of daily interactions (posts, comments, likes, shares, follows) must detect fake accounts and misinformation quickly, since delayed detection allows harmful content to go viral. This is a high-velocity, high-volume streaming problem where both structured signals (account metadata, graph relationships) and unstructured signals (text, images) must be analyzed in near real time.

Proposed Scalable Big Data Solution

- **Stream Ingestion:** All user interaction events are published to **Apache Kafka** topics partitioned by event type (post, comment, like, follow), giving the platform a scalable, fault-tolerant backbone capable of billions of events/day.
- **Stream Processing:** **Apache Flink** (preferred for its low-latency, exactly-once stateful processing) or Spark Structured Streaming performs windowed aggregation - e.g., posting frequency per account per minute, burst detection, and graph-based signals such as sudden follower spikes.
- **Feature Store & Graph Analysis:** A graph database (**Neo4j** or Spark GraphX/GraphFrames) models the social graph to detect fake-account clusters/botnets through community detection and abnormal connectivity patterns (e.g., many accounts created together, following identical target lists).
- **Machine Learning Layer:** Supervised classifiers (Random Forest / Gradient Boosting / deep neural networks) trained on labeled historical fraud cases score account authenticity using features such as account age, posting velocity, device fingerprints, and content similarity. NLP models (transformer-based classifiers, e.g., BERT-style) detect misinformation by analyzing text patterns, and are paired with fact-check knowledge bases for corroboration.
- **Real-Time Scoring & Action:** A low-latency model-serving layer (TensorFlow Serving / Spark Streaming ML pipeline) assigns a real-time risk score per event; high-risk content/accounts are automatically flagged, rate-limited, or queued for human moderator review.
- **Storage:** Raw events land in a data lake (S3/HDFS) for retraining; flagged entities and case history are stored in a NoSQL store (Cassandra/MongoDB) for fast moderator lookup.

Role of ML and Stream Processing

Stream processing frameworks (Flink/Spark Streaming) provide the low-latency backbone that lets the platform compute behavioural features (velocity, burstiness, graph anomalies) as events arrive rather than after the fact, which is essential because misinformation and fake-account activity must be intercepted within minutes, not hours. Machine learning turns these streaming features into an actionable authenticity/misinformation score, and models are continuously retrained on newly labeled data (including moderator feedback) to adapt to evolving fraud tactics - an approach known as an online/continual learning loop.

Security / Risk Considerations

- False positives must be minimized through human-in-the-loop review to avoid wrongly suspending genuine users; maintain an appeals workflow.
 - Protect user data used for ML features under privacy regulations (GDPR/local data protection law); avoid exposing PII in shared feature stores.
 - Guard against adversarial evasion - fraudsters adapting behaviour to bypass detection - via periodic model audits and ensemble/anomaly-detection redundancy rather than a single model.
-

Question 3: Government Data Platform Problem [8 Marks]

Industry Context

A national data platform integrating census, economic, and geospatial datasets across multiple government departments faces the classic big-data 'variety' and 'governance' challenge: heterogeneous formats and schemas, varying update frequencies, differing departmental ownership, and strict privacy/security mandates, all while needing to support cross-agency interoperability.

Proposed Governance and Data Management Strategy

- **Federated Data Architecture:** Adopt a data-mesh / federated model where each agency (census, economic, geospatial) retains ownership of its source data as a 'data product', but exposes it through standardized APIs and a shared metadata catalog rather than forcing a single centralized copy - improving both autonomy and interoperability.
- **Common Data Standards:** Define national interoperability standards (e.g., common geospatial reference system, standardized economic indicator schemas, open formats such as JSON/Parquet/GeoJSON) governed by a central data-standards committee so datasets from different departments can be joined reliably.
- **Metadata & Data Catalog:** Deploy a centralized data catalog (e.g., Apache Atlas, or a commercial catalog such as Collibra) providing data lineage, ownership, sensitivity classification, and search across agencies.
- **Storage & Processing:** A secure, government-controlled cloud/on-prem data lake (Hadoop HDFS or sovereign cloud storage) with tiered zones (raw/curated/published) and Spark/Hive for large-scale integration and analytics jobs such as census-economic cross-tabulation.
- **Data Governance Council:** Establish an inter-agency governance board that defines data-sharing agreements (MoUs), access policies, data quality standards, and dispute resolution for shared datasets.
- **Privacy & Security Controls:** Role- and attribute-based access control (ABAC), data anonymization and aggregation (k-anonymity/differential privacy) for sensitive census microdata, end-to-end encryption, and audit trails for every cross-agency query.

Solution Reasoning

A federated/data-mesh approach is more practical than a single monolithic warehouse for government because departments have independent legal ownership and update cycles; a shared catalog and common standards give the interoperability benefits of centralization without requiring every agency to give up control of its raw data.

Security / Risk Considerations

- Classify datasets by sensitivity (public, restricted, confidential) and apply differential privacy or aggregation before any cross-agency release of identifiable census data.
- Strong authentication (multi-factor) and fine-grained ABAC so a department can only query the fields it is authorized for, not the entire dataset.
- Maintain immutable audit logs for compliance/legal accountability, and a data breach response plan given the high sensitivity of national demographic data.

Question 4: Banking Fraud Detection Problem [8 Marks]

Industry Context

A bank processing millions of transactions per minute needs to detect fraud in real time while analyzing both streaming and historical transaction data, under strict low-latency and high-accuracy constraints - a textbook

big-data velocity and veracity challenge where a delayed or inaccurate decision has direct financial and customer-trust consequences.

Proposed Big Data Pipeline

- **Ingestion:** Transactions from ATMs, card networks, and online banking are streamed through **Apache Kafka**, chosen for its ability to sustain millions of events per minute with guaranteed delivery ordering per account/card.
- **Real-Time Processing:** **Apache Flink** (or Spark Structured Streaming) computes per-transaction features in milliseconds - e.g., transaction velocity in the last N minutes, deviation from the customer's typical spending pattern, geolocation jump detection (impossible-travel), and merchant-category anomalies.
- **Feature Store:** A low-latency feature store (Redis or a dedicated feature-store product) caches recent customer behaviour profiles so the model can score a transaction without a slow database round-trip.
- **Model Serving:** An ensemble of a supervised model (Gradient Boosted Trees / XGBoost, trained on labeled historical fraud data) and an unsupervised anomaly detector (Isolation Forest or Autoencoder, to catch novel fraud patterns not seen in training) scores each transaction in real time; scores above a threshold trigger automatic hold/step-up authentication, while borderline scores are queued for analyst review.
- **Historical/Batch Layer:** All transactions land in a data lake (HDFS/S3) with Spark batch jobs periodically retraining models on the latest confirmed fraud/non-fraud labels and computing longer-window behavioural baselines (e.g., 90-day spending profile).
- **Case Management:** Flagged transactions and investigation outcomes are stored in a relational/NoSQL store feeding a fraud-analyst dashboard, closing the feedback loop back into model retraining.

Technologies and Algorithms

Kafka + Flink provide the low-latency streaming backbone; XGBoost/Random Forest give high accuracy on structured transaction features; Isolation Forest/Autoencoders add unsupervised anomaly detection to catch previously unseen fraud typologies; and a rules engine layer (hard business rules, e.g., blocked countries) runs alongside the ML models as a fast, explainable safety net - a hybrid rules-plus-ML approach is standard industry practice because pure ML models can be slower to adapt to brand-new fraud rings than simple rules.

Security / Risk Considerations

- PCI-DSS compliance: tokenize/encrypt card data end-to-end; never store raw card numbers in the streaming or feature layers.
 - Model explainability (e.g., SHAP values) is required so declined transactions can be justified to regulators and customers.
 - Continuous model monitoring for concept drift, since fraud patterns evolve quickly; automatic alerting if false-positive/false-negative rates shift.
-

Question 5: Agriculture Smart Farming Problem [8 Marks]

Industry Context

A precision-agriculture system collects soil, weather, and crop data from distributed IoT sensors and drones across farms. The challenge is combining high-volume, high-variety sensor/imagery data from geographically dispersed and often low-connectivity sources into timely insights that help farmers improve yield and optimize water/fertilizer use.

Proposed Big Data Solution

- **Edge Layer:** Soil-moisture, temperature, and humidity sensors plus drone imagery are pre-processed at the edge (edge gateways/IoT hubs) to reduce raw data volume before transmission - important because rural connectivity is often limited/intermittent.
- **Ingestion:** Sensor telemetry is published via lightweight IoT protocols (**MQTT**) into **Apache Kafka**, aggregating data from many distributed farms into a central pipeline; drone imagery is uploaded in batches to a data lake when connectivity is available.
- **Stream Processing:** Spark Structured Streaming/Flink computes near-real-time alerts, e.g., soil moisture below threshold triggers an irrigation recommendation, or a sudden weather-forecast change triggers a spray-timing alert.
- **Storage:** Time-series data (soil/weather readings) is stored in a time-series database (InfluxDB/TimescaleDB); drone/satellite imagery is stored in a data lake (S3/HDFS) with geospatial indexing.
- **Analytics/ML:** Computer-vision models (CNNs) analyze drone/satellite imagery for crop-health indices (e.g., NDVI) to detect disease, pest stress, or nutrient deficiency early; regression/time-series models combine soil, weather, and historical yield data to recommend irrigation schedules, fertilizer dosage, and yield forecasts.
- **Delivery:** Insights are pushed to a farmer-facing mobile app/dashboard with simple actionable recommendations (e.g., 'irrigate Zone 3 tomorrow morning') rather than raw data, since end users are farmers, not data scientists.

How Analytics Supports Decision-Making

Combining time-series sensor analytics with image-based crop-health analytics lets the system move from reactive to predictive farming: instead of a farmer noticing a problem visually after damage has occurred, the system detects early stress signals (soil-moisture trends, NDVI drops) and recommends a targeted intervention for the specific field zone, optimizing water/fertilizer usage and improving yield while reducing waste.

Security / Risk Considerations

- Secure device authentication for thousands of distributed IoT sensors/drones to prevent spoofed readings (device certificates, mutual TLS).
- Data ownership clarity - farm-level data should remain the farmer's property, with clear consent for any aggregation used by agri-businesses or government schemes.
- Redundancy/offline buffering at the edge so temporary connectivity loss in rural areas does not cause data loss or missed irrigation alerts.

Note: Answers are written to align with the rubric criteria - Understanding of Industry Context (20%), Application of Big Data Concepts (25%), Solution Design (25%), Security/Risk Awareness (15%), and Clarity and Professionalism (15%) - for each of the five questions (8 marks each, totalling 40 marks).